DySR: A Dynamic Graph Neural Network Based Service Bundle Recommendation Model for Mashup Creation

Mingyi Liu, Zhiying Tu, *Member, IEEE*, Hanchuan Xu, Xiaofei Xu, *Member, IEEE*, and Zhongjie Wang, *Member, IEEE*

Abstract—An increasing number and diversity of services are available, which results in significant challenges to effectively reuse service during mashup creation. Many works have modeled the mashup creation problem as a service recommendation task and have achieved remarkable results. However, the performance of these methods can be further improved. The main problems affecting these methods include the constraints among recommended services, the evolution of services, and the semantic gap existing in services and mashups. In this article, we model the mashup creation problem as a service bundle recommendation task that is formally defined to address the constraints among recommended services. And then, a dynamic graph neural network based model called DySR is proposed to tackle the evolution of service and the semantic gap between services and mashups. In order to quantitatively measure how significant the semantic gap between mashups and services is, a measurement method of a semantic gap is given. With it, experiments show that to what extent DySR can reduce the semantic gap in the context of mashup creation. In addition, new evaluation metrics are introduced to overcome the preference for popular services in traditional service recommendations. Extensive experiments conducted on a real-world dataset from ProgrammableWeb, and the experiment results show that DySR outperforms existing state-of-the-art methods.

Index Terms—Dynamic graph neural networks, evolving service, mashup creation, semantic gap, service bundle recommendation

1 Introduction

With the rapid development of new technologies such as cloud, edge, and mobile computing, the number and diversity of available services are dramatically exploding. Services have become increasingly important to people's daily work and life. The increasing number and diversity of services bring significant challenges to effective service management and reuse. Consequently, recommending suitable services for developers based on requirements to help them reduce the burden of service selection in creating mashups becomes a non-trivial issue.

Inspired by the success of machine learning in other fields, recent works are widely constructing end-to-end models to solve mashup creation problems. These works normally model the mashup creation as a service recommendation problem, which refers to recommending a set of services based on these services' functions and history compositions to satisfy the user's explicit and implicit requirements [1]. These approaches can be categorized into natural language processing (NLP) based approaches [2], [3], [4], graph-based approaches [5], [6], [7], and hybrid approaches [8], [9]. However, ignoring the following factors limits the performance of their approaches and the application scenarios in reality:

(1) *Constrains among services:* Mashup creation requires a set of services to collaborate to achieve unique requirements, which means the recommended service should be complementary in function, and the recommended services should have a restriction to candidate services. However, most existing service recommendation studies independently recommend component services for a mashup, which leads to the neglect of the constraints among services. Service bundle recommendation aims to model these constraints into recommendation. Readers can refer to Section 3 to find the detailed difference.

(2) *Evolving services:* Services do not actively update their profile promptly in most cases. Existing approaches assume services' function and quality are static. However, we argue that the quality of services evolves, and the functionality of services evolves in a latent way as their social environment changes. For example, a service that other applications have not invoked for a long time may indicate that the service quality has degraded or that an alternative service with better similar functionality is available. Therefore, this service should be avoided when making service recommendations. *Readmill*[1] is an example of service function evolving [10], which is designed for "Social" as its profile says "*Readmill is*

• *The authors are with the Faculty of Computing, Harbin Institute of Technology, Harbin 150001, China. E-mail: {liumy, tzy_hit, xhc, xiaofei, rainy}@hit.edu.cn.*

Manuscript received 29 April 2022; revised 7 December 2022; accepted 2 January 2023. Date of publication 13 January 2023; date of current version 8 August 2023.
This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB3300700, in part by the National Science Foundation of China under Grants 61832004 and 61832014, and in part by the Key Research and Development Program of Heilongjiang Province, China under Grant 2022ZX01A11.
(Corresponding author: Zhongjie Wang.)
Recommended for acceptance by S. Russo.
Digital Object Identifier no. 10.1109/TSC.2023.3234293

1. https://www.programmableweb.com/api/readmill

1939-1374 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.

an online and mobile platform for readers to share information about..., allowing them to highlight and discuss sections of eBooks with other users...". The mashups that invoke *Readmill* are gradually moving from the "Social" domain to the "Book" domain, indicating that the social environment of *Readmill* is gradually evolving from "Social" to "Books".

(3) *Semantic gap between services and mashups:* There is a semantic gap between services and requirements representation, often overlooked in existing approaches (the rigorous definition can be found in Section 6.3.). The representation of mashups focuses on the functionality and value that the constructed mashup can provide to the user and is accompanied by an extensive domain vocabulary. In contrast, the representation of services is more liberal. Services can be described using natural language text, but they may describe their performance and input/output format in a technical-oriented language style. Services can also be represented using a set of properties, such as QoS, popularity. For example, *QuickMocker*[2] is a mashup where the requirement it satisfied is described as *"QuickMocker is an online API tool that allows to create your own public domain and a fake web services..."*. While its component service *Mocky*[3] are described as *"Mocky is available as a web console, downloadable software, or REST API. The API version of the service accepts calls issued in JSON and JSONP. The Mocky website is available in English, French, and Portuguese"*. These semantic gap make services and mashups exist in different semantic spaces. The arbitrary mixing of features in two different spaces in existing methods can significantly affect the performance of recommendations.

Besides the three major issues mentioned above, the following issues also need to be addressed in the recommendation model under the environment of massive services and explosive growth of mashups:

(1) *Long tail services:* The long tail is prevalent in generic recommendation problems [11] and is crucial in mashup creation. Recommending the well-known and frequently-used services (e.g., *Google Account*) is less meaningful than recommending services that developers are unfamiliar with but necessary for the mashup. Unfortunately, existing methods are misled by deficient evaluation metrics, which leads to the neglect of long-tail services.

(2) *Cold start of mashup and the scalability of model:* Existing state-of-the-art approaches leverage the interaction history between mashups and services to recommend missing services. They work well in typical situations like user-item recommendations and user-movie recommendations. The co-occurrence between users(mashups) and items(movies/services) can be repeated. However, in the application scenario of mashup creation, each mashup only appears once and without any interaction with existing services. The lack of such information decreases the performance of existing approaches. Also, due to the proliferation of services and mashups, the model should require as little information to be maintained as possible to guarantee the model's scalability.

Therefore, in this work, we propose a dynamic graph neural network-based service bundle recommendation model called DySR, a method that jointly addresses the above-mentioned three major issues. DySR also alleviates the problems of long-tail services, cold start, and model scalability in mashup creation. DySR model mashup creation as a service bundle recommendation task. Unlike traditional service recommendation tasks, service bundle recommendation explicitly considers the constraints among recommended services, which further solves the issue of long-tail services. DySR only requires the description of mashups in training and inference stages, which does not need to maintain other mashup-related information (e.g., mashup-service invoke matrix), which makes DySR adapt to cold start problem can be extended on a large scale. Different from the intuition of previous methods, which passing messages between mashups and services to reduce the semantic gap, DySR uses a graph neural network (GNN) to pass information between services to reduce the semantic gap. DySR also solves the problem of evolving service social by introducing dynamic graph neural network to learn service evolving representations.

The main contributions of this work are summarized as follows:

- *Problem Formulation* (Section 3): We formalize the mashup creation problem into a service bundle recommendation task, which explicitly integrates the constraints among recommended services into the recommendation results and further alleviates the long-tail service problem encountered in traditional mashup creation based on service recommendation.
- *Model* (Section 4): We propose DySR, an effective method for solving mashup creation problem by modeling the problem as service bundle recommendation task, and jointly addressing evolving service social and semantic gap issues. Moreover, DySR can work in cold start mashup and large-scale service situation.
- *Quantification Evaluation* (Sections 5.2 and 6): We introduce/adopt a variety of quantitative evaluation metrics from other communities to enrich the existing evaluation metrics and quantitatively support/correct some intuitive assumptions (e.g., the impact of evolving service social and semantic gap).
- *Effectiveness* (Section 6): Extensive experiments are conducted on the real-world dataset ProgrammableWeb. Experiment results shows that DySR achieves up to **6.12%**(**16.48%**) and **559.63%**(**551.68%**) better precision(discounted cumulative gain) and propensity-scored precision(discounted cumulative gain), respectively, than existing state-of-the-art service recommendation based mashup creation methods (Table 2).

The remainder of this paper is organized as follows: In Section 2, we introduce related work. In Section 3, we formally define the service bundle recommendation task and compare it with the traditional service recommendation task. In Section 4, we describe relevant details of the DySR model. In Section 5, we give the details of the experiment settings. In Section 6, we present the experiment results and discuss how DySR works. In Section 7, we analyze the sensitivity of DySR. Section 8 discusses possible opportunities for improvement of DySR and our further work. In the final section, we present the conclusion.

2. https://www.programmableweb.com/mashup/quickmocker
3. https://www.programmableweb.com/api/mocky

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.

2 Related Work

The use of service recommendations to satisfy the requirements of users creating new mashups has recently attracted a great deal of academic and industrial attention. Based on the type of information used in the model, existing service recommendation approaches can be divided into the following categories: NLP-based approaches that use the content information, graph-based approaches that use historical invocation information, and hybrid approaches that use both content and historical invocation information.

2.1 NLP-Based Approaches

The main idea of NLP-based approaches is to make recommendations by measuring the similarity between the description of the candidate services and the requirement. Keywords [12], [13] and TF-IDF [14] are used to match services to satisfy mashup creation. However, these matching procedures cannot really understand semantic and suffer from poor performance.

Ontology-based approaches annotate requirements and services with domain ontologies and use these ontologies to match high-level concepts or calculate their semantic similarities [15], [16], [17]. However, the lack of suitable domain ontologies and the huge manual annotation cost of ontology construction make ontology-based approaches difficult to use in real-world scenarios. Some studies attempt to use latent semantics to denote text features. Currently, the most common technique used to solve this problem in the field of service computing is the topic model [4], [18]. For example, Zhong et al. [19] refactor descriptions by using Author-Topic Model [20] to eliminate the gaps between mashups and services. However, small data volumes, high noise in data, and ignoring word orders make the performance of topic model-based methods unsatisfactory. With the great success of pre-trained language models (PLMs) and deep learning models in the NLP, some researchers have started using PLMs and deep learning to solve the service recommendation problem. For example, Bai et al. [21] designed a stacked denoising autoencoder (SADE) to extract features for the recommendation. However, experimental results show that the simple introduction of PLMs and deep learning does not significantly improve recommendation performance, possibly due to the inability of existing models to directly adapt to service recommendations and negative transfer [22].

The major drawback of the NLP-based approaches is the inability to leverage the social information between services, leading to recommended service bundles with combinations of services that cannot co-occur due to real-world constraints, despite being functionally compatible.

2.2 Graph-Based Approaches

Unlike NLP-based approaches, graph-based approaches make recommendations by mining information from historical interactions between services or requirements (users). For example, [5], [23], [24] design recommendation algorithms based on the feature learned on knowledge graph.

The graph-based approach usually works in conjunction with collaborative filtering (CF). For example, Chen et al. [25], [26] propose a neighborhood integrated matrix factorization approach to predict the quality of service (QoS) of candidate services. Chang et al. [6] designed a graph-based matrix factorization approach to predict QoS, then used the QoS to select services. Besides predicting QoS, graph is also applied to find similar users or services. For example, Maardji et al. [27] proposes a frequent pair mining method for mashup development. Qi et al. [28] adopts a hybrid random walk to compute the similarities between users or services, and a CF model is designed for service recommendation. [29], [30] build a heterogeneous information network using various information of services and mashups to measure the similarity between mashups and then use the user-based CF to rank candidate services.

Graph-based approaches cannot solve the cold start problem, as new requirements do not have any historical information.

2.3 Hybrid Approaches

Considering the complementarity of textual and graph information, a number of hybrid service recommendation approaches combining the two types of data have been proposed in recent years.

Li et al. [31] add the invocation relations between requirements and services to a latent Dirichlet allocation (LDA) model to enable the topic model to learn the relationship between services and requirements. [8], [14] incorporates data structure made up of service and their co-invocation records into LDA. Jain et al. [32], and Samanta et al. [33] use topic models and neighbor interaction probabilities to calculate similarity scores between services and requirements, then multiply these scores to rank candidate services. Deep learning-based approaches are becoming the mainstream hybrid methods. For example, Xiong et al. [34] integrate the invocation relations between services and requirements as well as their description similarity into a deep neural network (DNN). Chen et al. [35] propose a preference-based neural collaborative filtering [36] recommendation model, which uses multi-layer perceptron to capture the non-linear user-item relationships and obtain abstract data representation from sparse vectors. Ma et al. [9] utilize the powerful representation learning abilities provided by deep learning to extract textual features and features from various types of interactions between mashups and services. Wu et al. [37] propose a neural framework based on multi-model fusion and multi-task learning, which exploits a semantic component to generate representations of mashups and introduces a feature interaction component to model the feature interaction between mashups and services.

3 Problem Formulation

In this section, we formally state the differences between service recommendation and service bundle recommendation. We first give a generalized formulation of the two tasks and explain the advantages of service bundle recommendation by using toy examples. It should be noted that the main purpose of this section is to give a universal framework for service bundle recommendation, so some of the operations/modules are highly summarized and are not immediately bound under specific operations/modules. The instantiation of operations/modules will depend on specific scenarios. For example, for the mashup creation

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.

scenario in this paper, we will specify each operation/module involved in Eqs. (3) and (4) in Section 4.

3.1 Service Recommendation

Based on the discussion in Section 2, the existing end-to-end service recommendation models used for mashup creation can be uniformly modeled as

$$\begin{aligned} p(S|m,\Lambda) &= \prod_{s_i \in S} p(s_i|m,\Lambda) \\ &= p(s_0|m,\Lambda)p(s_1|m,\Lambda)\ldots p(s_k|m,\Lambda), \end{aligned} \tag{1}$$

where S is the recommended service set to construct the corresponding mashup m, Λ denotes additional information, such as historical invocation records, Quality of Service (QoS), etc., which does not constrain the relations between the services in the recommended service set S. Λ will be omitted for brevity when it is not ambiguous. $p(s_i|m)$ is the probability of mashup m invokes service s_i, which can be roughly represented as

$$p(s_i|m) = \frac{\phi(\psi_s(s_i), \psi_m(m))}{\sum_{s_j} \phi(\psi_s(s_j), \psi_m(m))}, \tag{2}$$

where $\psi_s(\cdot)$ and $\psi_m(\cdot)$ are encoders for service and mashup, respectively, and $\psi_s(\cdot)$ and $\psi_m(\cdot)$ can be related or unrelated to Λ. $\phi(\cdot)$ is a similarity measure function.

Under this kind of modeling, each service s in the recommended service set S is actually recommended independently. Such independent recommendations create redundancy in the functionality of the recommended service set. We use a toy example to illustrate this point. Suppose we have three services $\{s_a, s_b, s_c\}$, s_a and s_b belong to same category c_1 and have highly similar functions, while s_c belongs to category c_2. c_2 is very different from c_1. We also assume that the encoder $\psi_s(\cdot)$ is encoded based on the service category and functions, then it is reasonable to get $\psi_s(s_a) = [0.9, 0.8, 1, 0, 0]$, $\psi_s(s_b) = [0.8, 1, 0.9, 0, 0]$ and $\psi_s(s_c) = [0, 0, 0, 1, 0.7]$. Then we need to create a new mashup m that is a cross-board application of c_1 and c_2 with a slight bias towards c_1. And encoder $\psi_m(\cdot)$ is also based on category and functions, so we have $\psi_m(m) = [1, 1, 1, 0.9, 0.8]$. The dot product is selected as similarity function $\phi(\cdot)$ and the recommendation model are required to recommend 2 service for us. The existing service recommendation model will first calculate the probability of each service being invoked, $p(s_a|m) \approx 0.390, p(s_b|m) \approx 0.391, p(s_c|m) \approx 0.219$. Then s_b and s_a is recommended, while we want is s_c and one of s_a and s_b. Besides the redundancy of functions, when the encoders $\psi_s(\cdot)$ and $\psi_m(\cdot)$ related to frequency, existing methods are tend to recommend frequent invoked apis, however, frequent invoked apis is less informative, and such recommendations are often misleading under current evaluation metrics, which will be discussed in Section 5.2.

3.2 Service Bundle Recommendation

In order to satisfy the requirements of a mashup, the recommended services should not be independent, but constraints should exist between these services. We call such service recommendation with inter-service constraints in the recommended service set as service bundle recommendation. In this paper, inter-service constraints refer to functional requirements, which are expressed through text description (explicit) of service and the historical co-occurrence (implicit) between services. Service bundle recommendation can be formally modeled as maximizing the following probability:

$$\begin{aligned} p(S|m,\Lambda) &= \prod_{s_i \in S} p(s_i|m, \{s_0, s_1, \ldots, s_{i-1}\}, \Lambda) \\ &= p(s_0|m)p(s_1|m, \{s_0\}) \cdots p(s_k|m, \{s_0, \ldots, s_{k-1}\}), \end{aligned} \tag{3}$$

where $\{s_0, s_1, \ldots, s_{i-1}\}$ denotes the recommended services before recommend the ith service. It should be noted service bundle recommendation is different from service completion [38], [39], which requires a mashup to be given part of its component services and then predict the missing one. In contrast, service bundle recommendation aims to create a new mashup from scratch. Actually, the service completion task can be seen as a part of service bundle recommendation (the last term $p(s_k|m, \{s_0, s_1, \ldots, s_{k-1}\})$ in Eq. (3)). Eq. (3) consists of two kinds of conditional probabilities. One is the probability when the first recommended service is selected, and there is no limitation of already recommended services, which is the same as Eq. (2). The other is the probability when recommending a service with the constraints of previously recommended services

$$p(s_i|m, S') = \frac{\psi'(\phi_s(s_i), \phi_c(S'), \phi_m(m))}{\sum_{s_j \text{ s.t } s_j \notin S'} \psi'(\phi_s(s_j), \phi_c(S'), \phi_m(m))}, \tag{4}$$

where $S' = \{s_0, s_1, \ldots, s_{i-1}\}$. $\phi_c(\cdot)$ is an encoder to encode recommended services constraints. $\psi'(\cdot)$ is a similarity measure function.

The definition and modeling of service bundle recommendation can well address the above-mentioned problems in existing service recommendation models. We continue to use the toy example from the Section 3.1 to illustrate this point. With service bundle service recommendation, we begin with recommending the first service, which is same as service recommendation and s_b is selected. Then we choice the next service with $p(s|m, \{s_b\})$, suppose the $\phi_c(\cdot)$ is the sum of selected services feature and $\psi'(\mathbf{a}, \mathbf{b}, \mathbf{c}) = (\mathbf{c} - \mathbf{b})\mathbf{a}^T$. Under this condition the probability of recommend s_a and s_c are $p(s_a|m, \{s_b\}) \approx 0.110$ and $P(s_c|m, \{s_b\}) \approx 0.890$. Then s_c is recommended, which satisfied the requirement of the mashup.

4 Recommendation Methodology

In this section, we introduce the details of the DySR model. In Section 4.1 we first explain the overall framework of DySR. Next, we describe each component in detail.

4.1 Overall Framework

As shown in Fig. 1, the proposed DySR model consists of three components: a mashup transformation component, a service evolving component, and a bundle recommendation component. The mashup transformation component is designed to enhance the expression ability of the initial mashup representation. The service evolving component is the key to solving the semantic gap and evolving service social. The bundle recommendation component finally recommends related services

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.

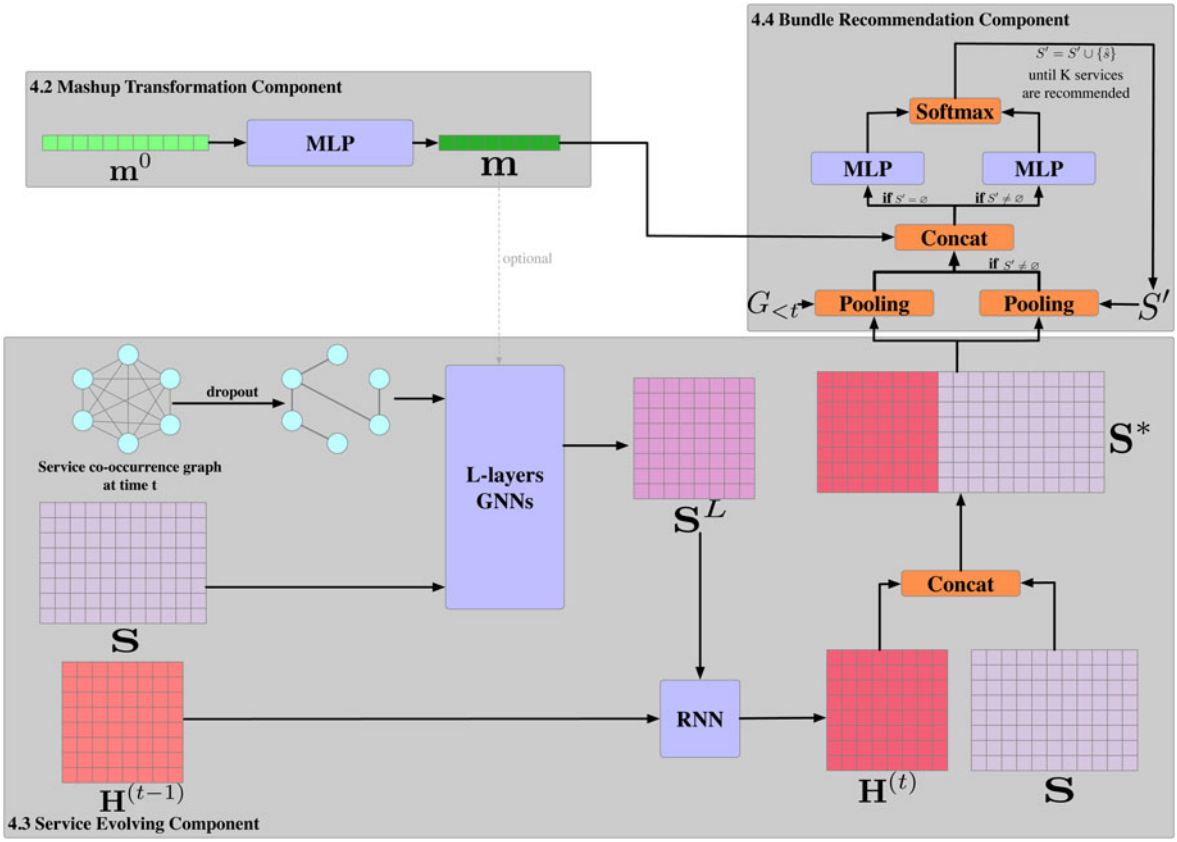


Fig. 1. Overview framework of DySR. Arrows denote the flow of data, the blue rectangular blocks indicate trainable neural network modules, the orange rectangular blocks indicate vector basis transformations.

based on the information provided by the previous component using the service bundle recommendation style.

During the training stage, the input of DySR mode is a mashup m that is satisfied as historical time t by a set of component services $S = \{s_0, s_1, \ldots, s_n\}$. While in the inference stage, the input of DySR model is a mashup m, and the output is a set of recommended component service $\hat{S} = \{\hat{s}_0, \hat{s}_1, \ldots, \hat{s}_m\}$. *The relaxed requirements for inputs (only the description for mashup is required) makes DySR suitable for cold start scenarios in mashup creation.*

During the entire training and inference stage, at any time t, we only need to maintain service static embeddings $\mathbf{S} \in \mathbb{R}^{N \times d_w}$, the service evolving embeddings $\mathbf{H}^{(t)} \in \mathbb{R}^{N \times d_e}$, and the invoked services set $G_{<t}$ before time t (These symbols will be detailed introduced in following sections). The maintenance of this information requires only $\mathcal{O}(N)$ space, where N is the number of candidate services. Compared to existing methods that need $\mathcal{O}(N^2)$ ($\mathcal{O}((N+M)^2)$, where M is the total number of mashups) space to maintain the service co-current graph (Mashup-service invoking graph), *DySR can be extended to large-scale service (bundle) recommendations in terms of space complexity.*

4.2 Mashup Transformation Component

For a given mashup m with a word sequences description $D_m = \{w_{m,1}, w_{m,2}, \ldots, w_{m,n}\}$, the initial mashup embedding can be denoted as

$$\mathbf{m}^0 = \frac{1}{|D_m|} \sum_{w \in D_m} \mathbf{v}_w, \tag{5}$$

where $\mathbf{v}_w \in \mathbb{R}^{d_w}$ is the embedding of word w, which can be obtained from a pre-trained language model (e.g., Word2Vec, Glove), word embeddings are frozen as tuning these embeddings is costly. Then, a multilayer perceptron (MLP) is used to enhance the expression ability of the mashup. The enhanced mashup embedding $\mathbf{m} \in \mathbb{R}^{d_m}$ can be denoted as

$$\mathbf{m} = \mathrm{MLP}(\mathbf{m}^0). \tag{6}$$

We deliberately design the mashup transformation component to be simple to accommodate the mashup creation scenarios in reality. In particular, we do not maintain information related to mashups because mashup creation is one-time, and the growth rate of mashups is extremely fast. Therefore, it is costly to maintain such one-time fast-growing information.

4.3 Service Evolving Component

We first use the description of the service to initialize the service static embeddings $\mathbf{S} \in \mathbb{R}^{N \times d_w}$. For each service s_i with description $D_s = \{w_{s,1}, w_{s,2}, \ldots, w_{s,n}\}$ the corresponding initial embedding is generated based on Eq. (5). After initialization, $\mathbf{S}$ is not associated with the service description and unlike the initialized mashup embeddings, $\mathbf{S}$ is trainable.

For a given mashup set M at time t, each mashup m's component services $S = \{s_0, s_1, \ldots, s_n\}$ can form a service co-occurrence graph $G_{m,t} = \{S, E\}$, where $E = \{(s_u, s_v) | s_u, s_v \in S\}$. The G_t is generated by combining all mashup's service

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.

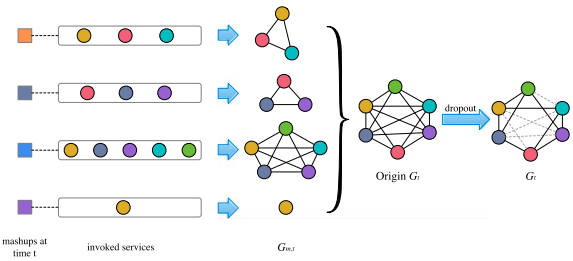


Fig. 2. Illustration of service co-occurrence graph construction and denoising.

co-occurrence graph at time t. In the origin G_t only a few of edges can reflect the dependency [40] and other edges can be treated as noise. Since there is no ground truth about the service dependency, we use a random edge drop strategy to mitigate the effect of noise (Section 7.2). Fig. 2 illustrate the aforementioned service co-occurrence graph construction and denoising process at time t.

Then, we utilize the graph neural network(GNN) to deal with the service co-occurrence graph. Specifically, we use GraphTransformer [41] to enhance the static embeddings of services. Given co-occurrence graph G_t, we update the static embedding of service s_i involved in G_t as follows:

$$\mathbf{s}_i^{(l,t)} = \mathbf{W}_s^l \mathbf{s}_i^{(l-1,t)} + \sum_{s_j \in \mathcal{N}^{(t)}(s_i)} \alpha_{i,j} \mathbf{W}_V^l \mathbf{s}_j^{(l-1,t)}, \tag{7}$$

where $\mathbf{s}_i^{(l,t)}$ denotes the service static embedding of service s_i learned by lth layer of the GraphTransformer by aggregating co-occurrence in G_t. We set the initial service static embedding $\mathbf{s}_i^{(0,t)}$ to $\mathbf{s}_i$ for each time t. $\mathbf{W}_s^l$ and $\mathbf{W}_V^l$ are learnable weight matrixes. $\mathcal{N}^{(t)}(s_i)$ is service s_i neighbors at time t. $\alpha_{i,j}$ is attention coefficient, which can be computed as follows:

$$\alpha_{i,j} = \frac{< \mathbf{W}_Q \mathbf{s}_i, \mathbf{W}_K \mathbf{s}_j >}{\sum_{u \in \mathcal{N}^{(t)}(s_i)} < \mathbf{W}_Q \mathbf{s}_i, \mathbf{W}_K \mathbf{s}_u >}, \tag{8}$$

where $< \mathbf{q}, \mathbf{k} > = \exp(\frac{\mathbf{q}^T \mathbf{k}}{\sqrt{C}})$ is exponential scale dot-product function. $\mathbf{W}_Q$ and $\mathbf{W}_K$ are two learnable parameters that are used to transform the source service embedding $\mathbf{s}_i$ and target service embedding $\mathbf{s}_j$ into query vector and key vector. It should be pointed out that any GNN layer can be used here, such as GCN [42], GAT [43]. *GraphTransformer is chosen here as it can be easily extended to support edge feature (mashup embedding) as additional input, which can help us to better explore the influencing factors of the semantic gap between services and mashups (Secion 6.3).*

We use C headers to model C different types of impact (e.g., strong dependencies, weak dependencies, functional overlaps, etc.) between services and use concatenation to compute the output embedding

$$\mathbf{s}_i^{(L,t)} = [\mathbf{s}_i^{(L,t),1}; \mathbf{s}_i^{(L,t),2}; \ldots; \mathbf{s}_i^{(L,t),C}]. \tag{9}$$

We stack L GNN layers, and layer normalization [44] is applied between two GNN layers.

To model the evolving of services, the DySR uses a recurrent neural network (RNN) to learn the service evolving embedding $\mathbf{h}_i^{(t)}$ as follows:

$$\mathbf{h}_i^{(t)} = \tanh(\mathbf{W}_{ih} \mathbf{s}_i^{(L,t)} + \mathbf{W}_{hh} \mathbf{h}_i^{(t-1)} + \mathbf{b}), \tag{10}$$

where $\mathbf{W}_{ih}$ and $\mathbf{W}_{hh}$ are trainable weight matrixes and $\mathbf{b}$ is the bias. We initial the service evolving embedding $\mathbf{h}_i^{(0)}$ to $\mathbf{0}$.

Finally, we concatenate the static and evolving embeddings to denote the service embedding at time t

$$\mathbf{s}_i^* = [\mathbf{h}_i^{(t)}; \mathbf{s}_i]. \tag{11}$$

4.4 Bundle Recommendation Component

We first obtain graph-level representation $\mathbf{g}^*$ via an element-wise mean pooling over the embeddings of all services $G_{<t}$ that have been invoked before time t

$$\mathbf{g}^* = \text{mean}(\{\mathbf{s}_i^* | s_i \in G_{<t}\}), \tag{12}$$

the graph-level embedding $\mathbf{g}^*$ can be treated as the service social background. Note that the pooling methods for $\mathbf{g}^*$ can be in various forms, such as max pooling, attention pooling and mean pooling. We use the mean pooling in our formulation, which tends to achieve better empirical performance.

Based on above embeddings, we can parameterize the first type term $p(s_0|m, \Lambda)$ in Eq. (3), where $\Lambda = G_{<t}$, as follows:

$$p(s_0|m, G_{<t}) = \text{softmax}(\text{MLP}([\mathbf{m}; \mathbf{g}^*])). \tag{13}$$

For the second type term $p(s_i|m, S,' G_{<t})$ in Eq. (3) can be parameterized as follows:

$$p(s_i|m, S,' G_{<t}) = \text{softmax}(\text{MLP}([\mathbf{m}; \mathbf{g}^*; \mathbf{c}^*])), \tag{14}$$

where $\mathbf{c}^*$ is the recommended service constraint embedding, which DySR obtains via an element-wise max pooling over recommended services S'

$$\mathbf{c}^* = \max(\{\mathbf{s}_i^* | s_i \in S'\}). \tag{15}$$

It should be noted, there have several different pooling strategies for Eq. (15), we use max pooling for better empirical performance.

4.5 Parameter Learning

Based on our problem formulation and modeling, the negative log-likelihood of a tuple $o = (m, S, t)$ (i.e., the *sorted* recommended services S for mashup m at time t) can be obtained as follows:

$$\begin{aligned} l(o) &= -\log p(S|m, G_{<t}) \\ &= -\log p(s_0|m, G_{<t}) - \sum_{i=1}^{|S|} \log p(s_i|m, S_i,' G_{<t}). \end{aligned} \tag{16}$$

During the training stage, the recommended services S is suppose to be sorted by their frequency, i.e., $Freq(s_i) > Freq(s_j) \Leftrightarrow i < j$ (Section 7.4).

We optimize DySR by minimizing the loss $\mathcal{L}$ for all mashups in the training set $\mathcal{D}$

$$\mathcal{L} = \sum_{o=(m,S,t) \in \mathcal{D}} l(o). \tag{17}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.

Based on above discussion, the parameter learning algorithm pseudo-code is shown in Algorithm 1.

Algorithm 1. Parameter Learning

Input: Training set $\mathcal{D}$ sorted by time step, number of epochs n, number of GNN layers L, edge drop ratio p.

for epoch = $1, \ldots, n$ **do**
  $G_{<t} = \{\}$
  $\mathbf{h}^{(0)} = \mathbf{0}$
  for $t \in$ Timestep($\mathcal{D}$) **do**
    Generate service co-occurrence graph G_t based on $\{(m, S, t) \in \mathcal{D}[t]\}$
    Random drop p edges from G_t
    /* executed in parallel */
    for $s_i \in Entities(G_t)$ **do**
      Compute $\mathbf{h}_i^{(t)}$ and $\mathbf{s}^*$ using Eqs. (10) and (11).
    end
    /* executed in parallel */
    for $(m, S, t) \in \mathcal{D}[t]$ **do**
      $S' = \emptyset$
      Compute $\mathbf{m}$ using Eq. (6)
      Compute $\mathbf{g}^*$ using Eq. (12)
      for $s \in S$ **do**
        if $S' == \emptyset$ **then**
          Compute $p(s|m, G_{<t})$ using Eq. (13)
        else
          Compute $\mathbf{c}^*$ using Eq. (15)
          Compute $p(s|m, S,' G_{<t})$ using Eq. (14)
        end
        $S' = S' \cup \{s\}$
      end
    end
    $G_{<t} = G_{<t} \cup Entities(G_t)$
    Compute the loss $\mathcal{L}_t$ for mashups in $\mathcal{D}[t]$ using Eq. (17)
    Optimize model parameters
  end
end

4.6 Complexity Analysis

For the mashup transformation component contains only one MLP, so the time complexity of this component is $\mathcal{O}(d_w d_m)$ ($d_w \ll N, d_m \ll N$).

For the service evolving component, we use the message passing framework to update the service static embedding, the time complexity of the GraphTransformer is $\mathcal{O}(LC_{max}|E|)$, where L is the layer number of the GraphTransformer layer, C_{max} is maximal header number in all GraphTransformer layers and $|E|$ is the edge number of service co-occurrence graph (after edge dropout). Here, the time complexities of attentions in GraphTransformer are not considered, as their time complexities are very low. It should be noted in the mashup creation scenario $|E|$ is much smaller than N. The time complexity of service evolving embedding module is equal to that of RNN (Eq. (10)). Suppose the input of the RNN are $N \times d_i$ and $N \times d_h$ matrices, then the time complexity is $\mathcal{O}(N(d_i + d_h))$. So the service evolving component is linear to the number of candidate services.

For the service bundle recommendation component consists of MLPs, and the final output dimension of these MLP is N. The complexity of this component is $\mathcal{O}(kd'N)$, where k is the number of recommended service and d' is input dimension.

Overall, *the time complexity of the DySR is linear to the number of candidate services, which proves that DySR can be applied to large-scale candidate services.*

TABLE 1
Statistic Information of ProgrammableWeb Dataset

Number of mashups	6200
Number of services	26631
Average number of services in the mashups	2.11
Average length of mashup's description	27.92
Average length of service's description	63.52

5 Experiment Settings

5.1 Dataset

We evaluate the proposed DySR model on the real-world *ProgrammableWeb* dataset, which is also the dataset used in existing service recommendation studies. The dataset is the largest online Web service registry. We use the dataset provided by [40]. We first remove mashups that do not invoke any services, and then remove stop words in mashups' and services' description. After these preprocess, the basic statistic information of the dataset is shown in Table 1. Fig. 3 shows the distribution of service invoked frequency, which demonstrates the existence of long-tail services in the mashup creation scenario.

We sorted the mashups by when they were created, then divided the training set, validation set, and test set according to the ratio of 7:1:2.

5.2 Metrics

In the mashup creation scenario, we recommend a few component services from a large number of services to build the corresponding mashups so that the *precision@k* and *DCG@k* metrics are defined for a recommended services $\hat{\mathbf{y}} \in \mathbb{R}^N$ and the ground truth invoked services label vector $\mathbf{y} \in \{0, 1\}^N$ as

$$P@k = \frac{1}{k} \sum_{l \in rank_k(\hat{\mathbf{y}})} \mathbf{y}_l \tag{18}$$

$$DCG@k = \sum_{l \in rank_k(\hat{\mathbf{y}})} \frac{\mathbf{y}_l}{\log(l+1)}, \tag{19}$$

where $rank_k(\mathbf{y})$ returns the k largest indices of $\mathbf{y}$ ranked in descending order.

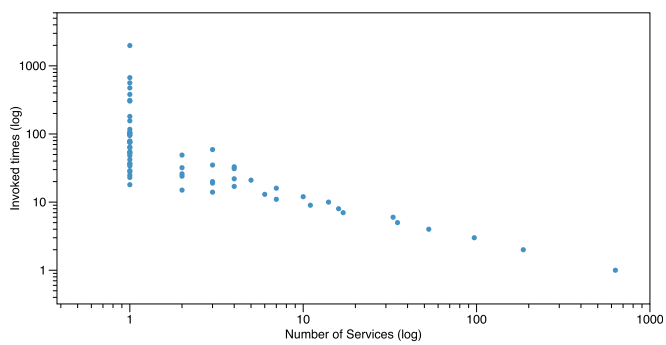


Fig. 3. The distribution of service invoked frequency. (Never invoked service number are not shown.).

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.

As we stated in Section 1, it is misleading to use only the two metrics $P@k$ and $DCG@k$. In mashup creation scenario, there are excessively frequently-used services (see Fig. 3), then high $P@k$ and $DCG@k$ can be obtained by simply recommend the frequently-used services. We introduce propensity-scored counterparts of *precision@k* and *DCG@k* metrics (*PSP@k* and *PSDCG@k*) [45]

$$PSP@k = \frac{1}{k} \sum_{l \in rank_k(\hat{\mathbf{y}})} \frac{\mathbf{y}_l}{p_l} \tag{20}$$

$$PSDCG@k = \sum_{l \in rank_k(\hat{\mathbf{y}})} \frac{\mathbf{y}_l}{p_l \log(l+1)}, \tag{21}$$

where p_l is the propensity score for service s_l, which can be calculated by the normalized service invoked frequency. Propensity-scored metrics place emphasis on performing well on tail services and give feeble rewards for recommending frequently-used services.

5.3 Baselines

To evaluate the effectiveness of model, we select two heuristic approaches and five state-of-the-art service recommendation approaches:

- *RandProb*: This approach randomly recommends k services from all candidate services based on the invocation frequency of services. *RandProb* is a heuristic approach.
- *Frequent*: This approach always recommends the top k frequently invoked services. *Frequent* is a heuristic approach.
- *MTFM* [37]: This approach jointly learns service recommendation task and mashup category prediction task by fusing CF-based model and NLP-based model. *MTFM* propose a semantic component to encode mashups and introduce a feature interaction component to model the interaction between mashups and services.
- *MLP*: This approach simply concatenates the mashup embedding and service embedding and then scores them using a multilayer perceptron. *MLP* is an efficient and straightforward baseline approach, but comparisons with it have been neglected in most existing work.
- *coACN* [46]: This approach consists of two components. The domain-level attention component is designed to refine mashup embeddings by drawing messages from related categories. The graph convolutional network component is established to excavate the service-composition graph and fuse structured information into service embeddings.
- *T2L2* [47]: This approach consists of three linear layers. The first two linear layers are used to align the representation space of services and mashups. The last linear layer is used to calculate the matching scores of services and mashups.
- *FCLSTM* [48]: This approach is an LSTM based model with two attention mechanisms: a functional attention mechanism that uses categories as prior knowledge to mine the function-related features of mashups and services. A contextual attention mechanism is used to help select the most appropriate services.

For a fair comparison, we initialize the word embeddings with the *Glove.6B.300 d* pre-trained word embeddings for all baselines and our DySR. The hyper-parameters of all baseline approaches are set according to the values given in their papers.

5.4 Implement Details

For DySR, we use *Glove.6B.300 d* pre-trained word embeddings to initialize mashups embedding $\mathbf{m}^0$ and the service static embedding $\mathbf{S}$, d_w is set to 300. The dimension d_m of enhanced mashup embedding $\mathbf{m}$ is set to 128. We set the output dimension of GNNs to 128, and we only stack 1 layer GNN by default. The edge dropout p is set to 0.4 by default. The GraphTransformer used in this paper is provided by PyG [49]. The learning rate is set to 0.0001. We conduct 5 independent experiments for each approach to preventing serendipity, and an early stop is applied to avoid over-fitting. We generate a time step for each of 32 mashups (batch size is set to 32). During the inference stage, the service evolving embeddings $\mathbf{h}^{(t)}$ will be updated after the inference.

All experiments are running on NVIDIA GeForce RTX 3080Ti with 12 GB memory.

6 Experiment Results

6.1 Overview

In this section, we give an overview performance comparison between our proposed DySR and baseline approaches. Table 2 shows the performance comparison of different approaches.

Both MTFM and FCLSTM do not work correctly under large-scale candidate services. MTFM encounters the problem of out of memory (OOM) during the training stage, caused by its feature interaction layer, fusion layer, and API specific task layer[37, Fig. 4], each of which requires $\mathcal{O}(N^2)$ trainable parameters. The inference speed of FCLSTM is extremely slow (about 60 times slower than other approaches.), and the performances ($P@5$ and $DCG@5$) are even worse than RandProb. It should be noted the high propensity-scored metrics obtained by FCLSTM is due to FCLSTM using the mashup categories in the inference stage, and it happens that there are several mashups and cold services with exactly the same cold categories in the test set, which causes these services to be selected. And normally, we assume the categories of the mashup are unknown at the time of mashup creation. MTFM and FCLSTM will be neglected in the following discussions.

The training and inference speed of coACN is the fastest among all benchmark approaches, but its $DCG@5$ value falls behind that of MLP and T2L2. Table 2 also shows that coACN, MLP, and T2L2 are very close in the rest three performance evaluation metrics ($P@5$, $PSP@5$, and $PSDCG@5$) and have a very limited improvement over Frequent, especially in the two propensity-scored metrics. *The high performance on $P@5$ and $DCG@5$ obtained by Frequent and the limited improvement of baselines on propensity-scored metrics prove that evaluation metrics of existing works are indeed somewhat misleading that the existing methods still tend to*

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Performance Comparison of Different Approaches

Model	P@5	PSP@5	DCG@5	PSDCG@5	train speed(s/epoch)	inference speed (s/epoch)
RandProb	0.011	2.144	0.034	6.899	-	-
Frequent	0.073	3.832	0.274	11.708	-	-
MTFM	OOM†	OOM†	OOM†	OOM†	OOM†	OOM†
MLP	0.096±0.002	4.924±0.213	0.361±0.006	14.977±0.511	1.661±0.038	1.467±0.004
coACN	0.093±0.000	4.531±0.000	0.319±0.002	15.998±0.121	**1.097±0.021**	**0.980±0.005**
L2T2	0.098±0.000	5.080±0.000	0.364±0.005	15.352±0.340	1.840±0.021	1.538±0.015
FCLSTM‡	0.003±0.001	64.776±11.973	0.011±0.002	225.795±42.409	2.136±0.049	62.382±0.015
DySR	**0.104*** ±0.004**	**33.509*** ±6.053**	**0.424*** ±0.015**	**104.256*** ±20.651**	2.367±0.025	1.233±0.022
(*improv.*)	(+6.12%)	(+559.63%)	(+16.48%)	(+551.68%)		

*In each column, the best result is **bolded**. * indicates that DySR significantly out performs the best baseline based on two-tail t-test (p-value < 0.05)*
†: *OOM indicates out of memory on NVIDIA 3080Ti with 12 GB memory.*
‡: *FCLSTM uses additional mashup label information in the inference, while other models use only the description of mashup in the inference.*

recommend popular services without actually understanding the requirements of mashups.

DySR achieves the best performance in all four evaluation metrics compared with all the baseline approaches. Specially, DySR achieves up to **6.12%**(**16.48%**) and **559.63%**(**551.68**%) better precision (DCG) and propensity-scored precision (DCG), respectively, than best baseline. The significant improvement in propensity-scored metrics shows that DySR does not favor popular services and solves the problem of long-tail services well compared to the baseline approaches. While the improvement of DySR over $P@5$ is not as obvious as $PSP@5$, the $PSP@5$ value can also illustrate that the services recommended by DySR are significantly different from those recommended by baselines, and DySR can recommend the non-popular and necessary services, which has much more significance in reducing the burden of developers during mashup creation.

In terms of training speed, DySR takes a longer time per epoch than all baselines due to the different design of loss functions for service recommendation task and service bundle recommendation task. The baseline methods generate negative samples by sampling the n ($n \ll N$) candidate services and then computes them in parallel with the positive samples, while DySR does not need to perform negative sampling and computes the probability of each service in the mashup composition services sequentially, with all candidate services considered in each computation step (Section 4.4). It should be noted that negative sampling is usually difficult to gather informative negative samples, so the number of convergence epochs required for baseline approaches is much higher (about 5x) than that required by DySR to converge, so *the total training time of DySR is actually lower than that of the baseline approaches*. Since there is no need to calculate the score of each mashup-service pair separately, DySR has an advantage in inference speed.

Based on the above discussions, we demonstrate the effectiveness and efficiency of DySR w.r.t. existing SOTA approaches and show that DySR can well address the first major problems stated in Section 1. Furthermore, we would like to gain insight into 1) whether the remaining two problems (evolving services and semantic gap) stated in Section 1 actually exist in the mashup creation scenarios; 2) what impact they have on mashup creation; 3) how and to what extent DySR solves these problems. The next two subsections will discuss these in detail.

6.2 Evolving Services

To demonstrate that the evolution of services has an impact on mashup creation, we use the validation set and test set to simulate the difference in the performance of different models in the near and far future. We also construct two ablation experiments to explore how DySR handles service evolution:

- *DySR-w/o-evolving*: This variant removes the service evolving embeddings from DySR, i.e., remove Eq. (10) and make $\mathbf{s}_i^* = \mathbf{s}_i$ in Eq. (11).
- *DySR-w/o-inferenceUpdate*: This variant does not update the service evolving embeddings after inference.

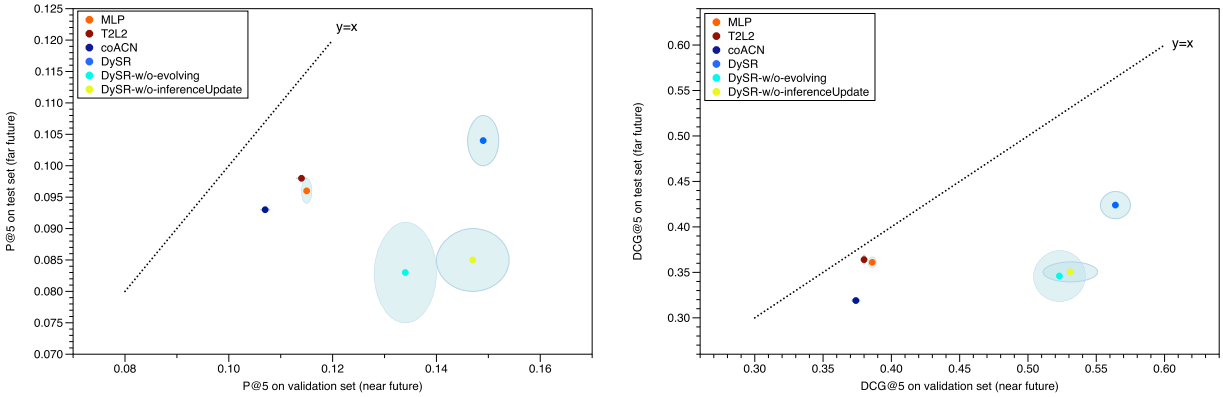
Fig. 4 shows the performance of the different approaches on the near and far future, where the x-axis is the performance on the near future and the y-axis is the performance on the far future.

All approaches are below the $y = x$, which means that all approaches have higher performance in the near future than far future, *This confirms that the evolution of services does have an impact on mashup creation*, and as time moves forward, some of the patterns that exist in history change, causing performance degradation.

From Fig. 4 we can also find that the baseline approaches are closer to the line $y = x$ compared to DySR and its variants, which means that the baseline approaches cannot perceive the evolution of the service and tend to recommend popular historical services. Additionally, we find that DySR and its variants perform much better than the baseline approaches in near future, which illustrates the superiority of modeling the mashup creation as a service bundle recommendation problem over modeling it as a traditional service recommendation problem.

Compared to DySR-w/o-evolving and DySR-w/o-inferenceUpdate, DySR has significantly improved performance both in the near and far future. This illustrates that the service evolving component of DySR can indeed handle the evolution of service. DySR supports continuous updating of service embeddings during the inference stage, which makes DySR more robust.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.



(a) P@5 (b) DCG@5

Fig. 4. Performance of the different approaches on the near and far future.

6.3 Semantic Gap Between Services and Mashups

Semantic gap between services and mashups has been introduced in many works [10], [47], [48]. However, these works only illustrate the existence of the semantic gap through carefully selected examples, and yet how to illustrate the existence of the semantic gap between services and mashups theoretically and experimentally has not been addressed, let alone quantitatively evaluate how large the size of semantic gap in the original representation and how much the recommendation model actually reduces the semantic gap.

The semantic gap between services and mashups means that the service representation space is obtained through $\phi_s(\cdot)$ and the mashup representation space obtained through $\phi_m(\cdot)$ differ in their *expressiveness*. The difference in expressive ability is hard to be evaluated on their values (e.g., PCA visualization), but rather on the performances they achieve when applied to the same downstream task [50], [51], [52]. More straightforwardly, for a downstream task $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the downstream task can be shared by both the API and mashup. If there is no semantic gap or a small semantic gap, then for given test services (mashups) $\mathcal{D}_t$, the model f_a trained on the service training set $\mathcal{D}_s$ should have similar *results (NOT performance.)* as the model f_m trained on the mashup training set $\mathcal{D}_m$. Thus, we can define the semantic gap between service and mashups as follows:

$$sg = \frac{1}{|\mathcal{D}_t|} \sum_{i \in \mathcal{D}_t} JSD(f_a(\phi(i)) || f_m(\phi(i))), \tag{22}$$

where $\phi(i)$ is one of $\phi_m(i)$ and $\phi_s(i)$, depending on whether i is a mashup or a service. $JSD(\cdot)$ is Jensen-Shannon distance between tow probability arrays, which is the square root of the Jensen-Shannon divergence [53]. Jensen-Shannon divergence is a symmetrsized and smoothed version of the Kullback–Leibler divergence (K-L divergence, denoted with D_{KL}). Jensen-Shannon distance between two probability vectors $\mathbf{q}$ and $\mathbf{p}$ is defined as

$$JSD(\mathbf{p}||\mathbf{q}) = \sqrt{\frac{D_{KL}(\mathbf{p}||\mathbf{o}) + D_{KL}(\mathbf{q}||\mathbf{o})}{2}} \tag{23}$$

$$D_{KL}(\mathbf{p}||\mathbf{o}) = \sum_{i=0}^{|\mathbf{p}|} \mathbf{p}_i \ln(\frac{\mathbf{p}_i}{\mathbf{o}_i}), \tag{24}$$

where $\mathbf{o}$ is the pointwise mean of $\mathbf{q}$ and $\mathbf{p}$.

To measure the semantic gap between services and mashups and the extent to which the different models address this semantic gap. We choose to predict the primary labels of services and mashups as a downstream task. For the downstream task model, we choose SVM for both mashups and services. For each model, we select the service and mashup embeddings that are used in the last layer of the model for recommendation. The results are shown in Fig. 5.

We set two baseline semantic gaps in Fig. 5, one marked as Glove, in which both the service and mashup embeddings are obtained from pre-trained *Glove.6B.300 d* word embeddings. The other is marked as Random, in which the mashup embeddings are obtained from pre-trained *Glove.6B.300 d* word embeddings, and the service embeddings are randomly generated. The results show that the semantic gap of Glove is significantly smaller than Random, which is consistent with common sense.

$$\mathbf{s}_i^{(l,t)} = \mathbf{W}_s^l \mathbf{s}_i^{(l-1,t)} + \sum_{s_j \in \mathcal{N}^{(t)}(s_i)} \alpha_{i,j} (\mathbf{W}_V^l \mathbf{s}_j^{(l-1,t)} + \mathbf{W}_E^l \mathbf{e}_{ij}^t) \tag{25}$$

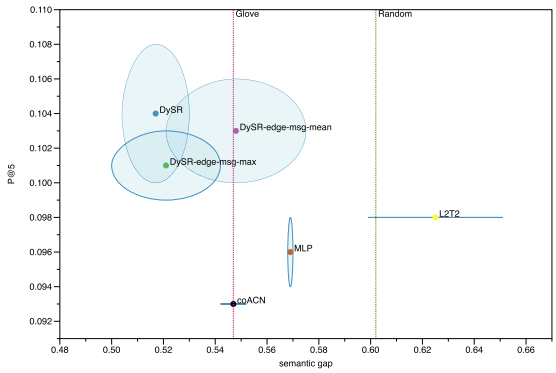
$$\alpha_{i,j} = \frac{< \mathbf{W}_Q \mathbf{s}_i, \mathbf{W}_K \mathbf{s}_j + \mathbf{W}_E^l \mathbf{e}_{ij}^t >}{\sum_{u \in \mathcal{N}^{(t)}(s_i)} < \mathbf{W}_Q \mathbf{s}_i, \mathbf{W}_K \mathbf{s}_u + \mathbf{W}_E^l \mathbf{e}_{ij}^t >}. \tag{26}$$

Fig. 5 shows that our proposed DySR can indeed significantly reduce the semantic gap. It is surprising that all of the baseline approaches do not actually reduce the semantic gap. Especially for T2L2, which expects to reduce the semantic gap by propagating the mashup information to services, however, the experimental results show that this intuitive approach widens the semantic gap. To verify this point, we designed two ablation experiments:

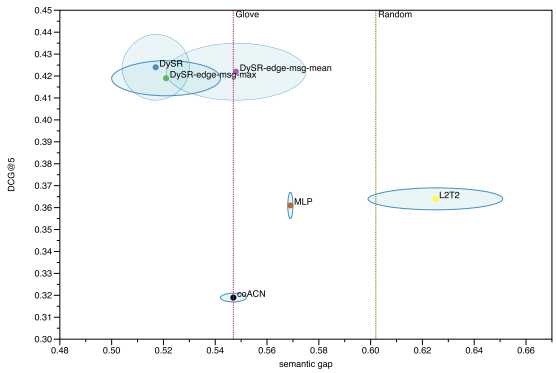
- *DySR-edge-msg-mean*: In this variant, we replace the Eqs. (7) and (8) in DySR to Eqs. (25) and (26). The edge message is generated by using element-wise mean pooling on mashups $\mathbf{e}_{ij}^t = \text{mean}(\mathbf{m}_a | m_a \in \mathcal{D}[t]$ and $s_i, s_j \in S_{m_a})$.
- *DySR-edge-msg-max*: In this variant, we replace the Eqs. (7) and (8) in DySR to Eqs. (25) and (26). The edge message is generated by using element-wise mean pooling on mashups $\mathbf{e}_{ij}^t = \max(\mathbf{m}_a | m_a \in \mathcal{D}[t]$ and $s_i, s_j \in S_{m_a})$.

Fig. 5 illustrates that *directly propagating information from the mashup to service increases the semantic gap*.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.



(a) Semantic gap and P@5



(b) Semantic gap and DCG@5

Fig. 5. The relations between model performance and semantic gap.

We also find that coACN has the smallest semantic gap among all baseline approaches, and we believe this is due to the benefit of GNN. To verify this point, we explore the effect of the different number of GNN layers on the semantic gap in DySR. We test the number of GNN layers $L = \{1, 2, 3, 4\}$, the results are shown in Fig. 6b. It is clear that the semantic gap is decreasing as the number of GNN layers rises.

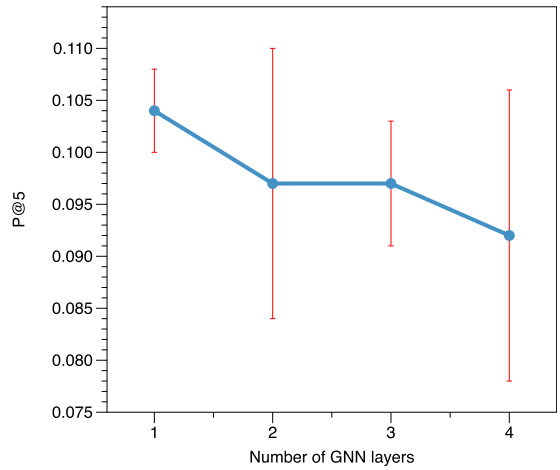
It should be noted that *reducing the semantic gap is helpful to improve the recommendation performance of the model, but the semantic gap is not a decisive factor, as 1) in Fig. 5, coACN has the smallest semantic gap among all baseline approaches, while the performance is not and 2) in Fig. 6, with the increment of GNN layers, the semantic gap of DySR is decreasing and performance is also slowly decreasing. The recommendation performance in the mashup creation scenario is affected by a combination of factors.*

7 Analysis of the Sensitivity of Hyper Parameters

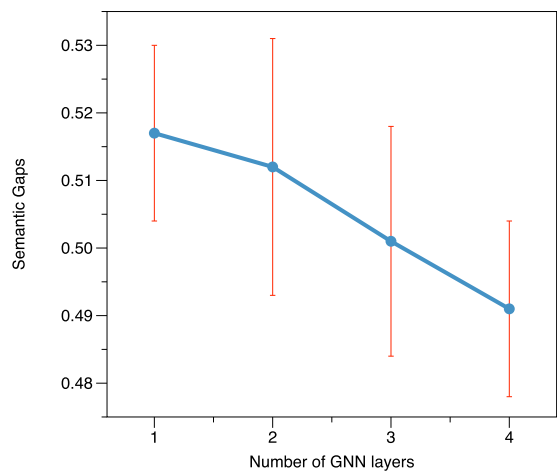
The proposed DySR involves a number of parameters that may affect its performance. In this section we discuss the effects of different parameters.

7.1 Learning Rate

We first examine how the learning rate affect the performance of DySR and select the learning rate from $\{0.001, 0.0001, 0.00005, 0.00001\}$. As Fig. 7 shows, the best performance is achieved by learning rate of 0.0001 and 0.00005. Using a too small learning rate will make the DySR converge too slowly and still not fully converge when the maximum epoch is reached, while a too large learning rate will hurt the model performance. For DySR, the recommended learning rate is between 0.00005 and 0.0001.



(a) P@5



(b) Semantic gaps

Fig. 6. The effect of different number of GNN layers on semantic gap and performance in DySR.

7.2 Edge Dropout Ratio

As we stated in Section 4.3, most co-occurring services do not have an actual dependency but are a kind of noise, and we mitigate this problem by randomly dropping out co-occurring edges. The edge dropout ration affects how much noise is eliminated and how much risk there is of eliminating useful information.

Fig. 8 reports the performance of DySR as the edge dropout ratio p is set to 0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8 and 0.9. Obviously, the performance of DySR increases as the edge dropout ratio increases. However, when the dropout ratio is too high, there is an increased risk of valuable information being dropped out, which leads to a significant performance degradation (e.g., $p = 0.8$).

7.3 Attention Headers C

The attention headers C in Eq. (9) can model C different service collaboration patterns, and these different service collaboration patterns introduce different constraints among

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.

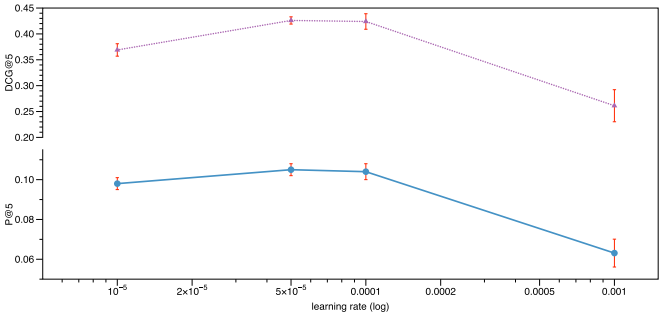


Fig. 7. Influence of learning rate.

services, further affecting the propensity-scored performance ($PSP@5$ and $PSDCG@5$).

Fig. 9 reports the propensity-scored performance of DySR as the attention headers C is set to 1, 2, 4, 8 and 16. The best propensity-scored performance is achieved by an attention headers of 8, while using a large attention headers of 16 hurts the propensity-scored performance. The results also demonstrate that multiple types of service collaboration patterns do exist and that modeling these patterns can effectively improve the propensity-scored performance of the model.

7.4 Order of the Service Set S

The order of the recommended service set S in Eq. (17) determines the order of dependencies between the recommended services and also reflects the order in which developers select services during the mashup creation.

We tested three types of sorting methods based on the frequency of service invocations, i.e., descending, ascending and random. Table 3 shows the results of these three sorting methods. The results show that DySR is sensitive to the order of recommended service set S, especially in terms of propensity-scored performance. The descending order obtains the best performance by simulating the developer's intuitive selection order (i.e., selecting familiar ones first).

8 Discussion & Future Work

8.1 Introduce Service Dependencies

In the current version of DySR, we use service co-occurrence graph G_t to describe the relation between services due to data limitations, but this graph contains a lot of noise. Although we tried to reduce the noise by random masking

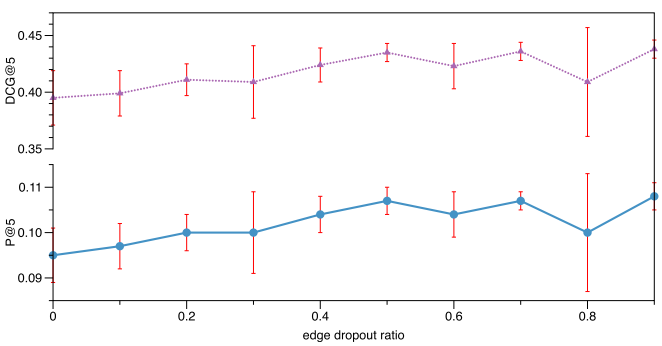


Fig. 8. Influence of edge dropout ratio.

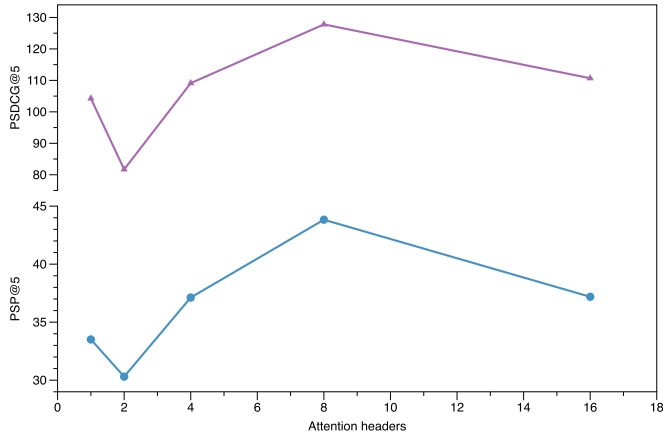


Fig. 9. Influence of attention headers.

edges, we believe there are two more ways further to filter out the valuable service dependencies/flows:

- *Labeling mashup workflows:* The mashup is not simply a collection of services but a workflow of services. If the workflow of the component services of mashups can be labeled, then the noise problem in the service co-occurrence graph mentioned above can be avoided.
- *Learning service dependencies/flows:* Labeling mashup workflows requires large human resources, which may be unacceptable when the data volume is too large. So another way is to try to learn the service dependencies/flows from the service co-occurrence graph. This can be converted into a binary classification problem for co-occurrence edges, and the Neural Relational Inference (NRI) [54] would be applicable to this scenario. However, how to reduce the training cost will be another challenge.

8.2 Combine With Service Composition

The original search space for recommending k services for a mashup in the service bundle recommendation problem is $\mathcal{O}(N^k)$. To achieve online recommendation, we use a greedy strategy to reduce the search space to $\mathcal{O}(kN)$ in the inference stage, at the cost that the recommended result is not the optimal solution but the suboptimal solution. In the case of offline recommendation, the optimal solution can be searched using the service combination methods [55](e.g., S-ABC [56]) to future improve the performance of DySR.

TABLE 3
Influence of Order of the Service Set S

Model	P@5	PSP@5	DCG@5	PSDCG@5
DySR-descending	0.104 (±0.004)	**33.509 (±6.053)**	**0.424 (±0.015)**	**104.256 (±20.651)**
DySR-ascending	0.103 (±0.005)	19.425 (±5.456)	0.376 (±0.028)	83.883 (±29.740)
DySR-random	**0.105 (±0.002)**	18.778 (±5.912)	0.423 (±0.008)	60.512 (±18.629)

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.

9 Conclusion

Recommending suitable services when developers create new mashups can significantly reduce the burden on the developer. Towards an effective recommendation, this paper first models the mashup creation problem as a service bundle recommendation task, which can naturally model the constraints among services explicitly and thus further solve the long-tail service problem. Then, we propose an end-to-end deep learning model called DySR. DySR solves the service evolution problem and eliminates the semantic gap between services and mashups by introducing a service evolving component. Furthermore, DySR only requires the description of mashups in both training and inference stages and does not need to maintain other mashup-related information (e.g., mashup-service invoke matrix), which makes DySR adapt to cold start and can be extended on a large scale. Experiments on a real-world dataset demonstrated that the proposed approach outperforms several state-of-the-art service recommendation methods.

In addition to the performance improvements, this paper refines the evaluation metrics in the mashup creation scenario and quantitatively evaluates the example-based assumptions from previous work. We also give the future opportunities and challenges that exist based on DySR.

References

[1] W. Zhaohui, D. Shuiguang, and W. Jian, *Chapter 6 - Service Recommendation Service Computing*. Boston, MA, USA: Academic Press, 2015, pp. 133–176.

[2] S. Faieq, A. Front, R. Saidi, H. El Ghazi, and M. D. Rahmani, "A context-aware recommendation-based system for service composition in smart environments," *Service Oriented Comput. Appl.*, vol. 13, no. 4, pp. 341–355, 2019.

[3] Y. Lei, Z. Jiantao, Z. Junxing, W. Fengqi, and W. Juan, "Time-aware Semantic Web service recommendation," in *Proc. IEEE Int. Conf. Services Comput.*, 2015, pp. 664–671.

[4] C. Lin, A. Kalia, J. Xiao, M. Vukovic, and N. Anerousis, "NL2API: A framework for bootstrapping service recommendation using natural language queries," in *Proc. IEEE Int. Conf. Web Serv.*, 2018, pp. 235–242.

[5] S. Hu, Z. Tu, Z. Wang, and X. Xu, "A POI-sensitive knowledge graph based service recommendation method," in *Proc. IEEE Int. Conf. Services Comput.*, 2019, pp. 197–201.

[6] Z. Chang, D. Ding, and Y. Xia, "A graph-based QoS prediction approach for web service recommendation," *Appl. Intell.*, vol. 51, pp. 6728–6742, 2021.

[7] Z. Cao, X. Qiao, S. Jiang, and X. Zhang, "An efficient knowledge-graph-based web service recommendation algorithm," *Symmetry*, vol. 11, no. 3, 2019, Art. no. 392.

[8] Z. Gao et al., "SeCo-LDA: Mining service co-occurrence topics for recommendation," in *Proc. IEEE Int. Conf. Web Serv.*, 2016, pp. 25–32.

[9] Y. Ma, X. Geng, and J. Wang, "A deep neural network with multiplex interactions for cold-start service recommendation," *IEEE Trans. Eng. Manag.*, vol. 68, no. 1, pp. 105–119, Feb. 2021.

[10] B. Bai, Y. Fan, W. Tan, and J. Zhang, "SR-LDA: Mining effective representations for generating service ecosystem knowledge maps," in *Proc. IEEE Int. Conf. Services Comput.*, 2017, pp. 124–131.

[11] R. S. Sreepada and B. K. Patra, "Enhancing long tail item recommendation in collaborative filtering: An econophysics-inspired approach," *Electron. Commerce Res. Appl.*, vol. 49, 2021, Art. no. 101089.

[12] Q. He et al., "Keyword search for building service-based systems," *IEEE Trans. Softw. Eng.*, vol. 43, no. 7, pp. 658–674, Jul. 2017.

[13] Q. He et al., "Efficient keyword search for building service-based systems based on dynamic programming," in *Proc. Int. Conf. Service-Oriented Comput.*, 2017, pp. 462–470.

[14] B. Xia, Y. Fan, W. Tan, K. Huang, J. Zhang, and C. Wu, "Category-aware API clustering and distributed recommendation for automatic mashup creation," *IEEE Trans. Services Comput.*, vol. 8, no. 5, pp. 674–687, Sep./Oct. 2015.

[15] M. Al-Hassan, H. Lu, and J. Lu, "A semantic enhanced hybrid recommendation approach: A case study of e-government tourism service recommendation system," *Decis. Support Syst.*, vol. 72, pp. 97–109, 2015.

[16] N. Karthikeyan et al., "Fuzzy service conceptual ontology system for cloud service recommendation," *Comput. Elect. Eng.*, vol. 69, pp. 435–446, 2018.

[17] R. A. Rupasingha and I. Paik, "Alleviating sparsity by specificity-aware ontology-based clustering for improving web service recommendation," *IEEE Trans. Elect. Electron. Eng.*, vol. 14, no. 10, pp. 1507–1517, 2019.

[18] Q. Gu, J. Cao, and Y. Liu, "CSBR: A compositional semantics-based service bundle recommendation approach for mashup development," *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3170–3183, Nov./Dec. 2022.

[19] Y. Zhong, Y. Fan, W. Tan, and J. Zhang, "Web service recommendation with reconstructed profile from mashup descriptions," *IEEE Trans. Automat. Sci. Eng.*, vol. 15, no. 2, pp. 468–478, Apr. 2018.

[20] M. Rosen-Zvi, T. Griffiths, M. Steyvers, and P. Smyth, "The author-topic model for authors and documents," 2012, *arXiv:1207.4169*.

[21] B. Bai, Y. Fan, W. Tan, and J. Zhang, "DLTSR: A deep learning framework for recommendations of long-tail web services," *IEEE Trans. Services Comput.*, vol. 13, no. 1, pp. 73–85, Jan./Feb. 2020.

[22] Z. Wang, Z. Dai, B. Póczos, and J. Carbonell, "Characterizing and avoiding negative transfer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11293–11302.

[23] H. Wang, Z. Wang, S. Hu, X. Xu, S. Chen, and Z. Tu, "DUSKG: A fine-grained knowledge graph for effective personalized service recommendation," *Future Gener. Comput. Syst.*, vol. 100, pp. 600–617, 2019.

[24] H. Mezni, D. Benslimane, and L. Bellatreche, "Context-aware service recommendation based on knowledge graph embedding," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 11, pp. 5225–5238, Nov. 2022.

[25] Z. Zheng, H. Ma, M. R. Lyu, and I. King, "Collaborative web service QoS prediction via neighborhood integrated matrix factorization," *IEEE Trans. Services Comput.*, vol. 6, no. 3, pp. 289–299, Third Quarter 2013.

[26] X. Chen, X. Liu, Z. Huang, and H. Sun, "RegionKNN: A scalable hybrid collaborative filtering algorithm for personalized web service recommendation," in *Proc. IEEE Int. Conf. Web Serv.*, 2010, pp. 9–16.

[27] A. Maaradji, H. Hacid, R. Skraba, and A. Vakali, "Social web mashups full completion via frequent sequence mining," in *Proc. IEEE World Congr. Serv.*, 2011, pp. 9–16.

[28] L. Qi, Z. Zhou, J. Yu, and Q. Liu, "Data-sparsity tolerant web service recommendation approach based on improved collaborative filtering," *IEICE Trans. Inf. Syst.*, vol. 100, no. 9, pp. 2092–2099, 2017.

[29] F. Xie, J. Wang, R. Xiong, N. Zhang, Y. Ma, and K. He, "An integrated service recommendation approach for service-based system development," *Expert Syst. Appl.*, vol. 123, pp. 178–194, 2019.

[30] T. Liang, L. Chen, J. Wu, H. Dong, and A. Bouguettaya, "Meta-path based service recommendation in heterogeneous information networks," in *Proc. Int. Conf. Service-Oriented Comput.*, 2016, pp. 371–386.

[31] C. Li, R. Zhang, J. Huai, and H. Sun, "A novel approach for API recommendation in mashup development," in *Proc. IEEE Int. Conf. Web Serv.*, 2014, pp. 289–296.

[32] A. Jain, X. Liu, and Q. Yu, "Aggregating functionality, use history, and popularity of APIs to recommend mashup creation," in *Proc. Int. Conf. Service-Oriented Comput.*, 2015, pp. 188–202.

[33] P. Samanta and X. Liu, "Recommending services for new mashups through service factors and top-k neighbors," in *Proc. IEEE Int. Conf. Web Serv.*, 2017, pp. 381–388.

[34] R. Xiong, J. Wang, N. Zhang, and Y. Ma, "Deep hybrid collaborative filtering for web service recommendation," *Expert Syst. Appl.*, vol. 110, pp. 191–205, 2018.

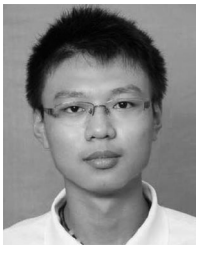
[35] L. Chen, A. Zheng, Y. Feng, F. Xie, and Z. Zheng, "Software service recommendation base on collaborative filtering neural network model," in *Proc. Int. Conf. Service-Oriented Comput.*, 2018, pp. 388–403.

[36] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proc. 26th Int. Conf. World Wide Web*, 2017, pp. 173–182.

[37] H. Wu, Y. Duan, K. Yue, and L. Zhang, "Mashup-oriented web API recommendation via multi-model fusion and multi-task learning," *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3330–3343, Nov./Dec. 2022.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.

[38] K. Huang, Y. Fan, and W. Tan, "Recommendation in an evolving service ecosystem based on network prediction," *IEEE Trans. Automat. Sci. Eng.*, vol. 11, no. 3, pp. 906–920, Jul. 2014.
[39] Y. Ma, X. Geng, J. Wang, K. He, and D. Athanasopoulos, "Deep learning framework for multi-round service bundle recommendation in iterative mashup development," in *Proc. CAAI Trans. Intel Tech.*, 2022.
[40] M. Liu, Z. Tu, Y. Zhu, X. Xu, Z. Wang, and Q. Z. Sheng, "Data correction and evolution analysis of the ProgrammableWeb service ecosystem," *J. Syst. Softw.*, vol. 182, 2021, Art. no. 111066.
[41] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, "Masked label prediction: Unified message passing model for semi-supervised classification," in *Proc. 30th Int. Joint Conf. Artif. Intell.*, 2021, pp. 1548–1554.
[42] S. Zhang, H. Tong, J. Xu, and R. Maciejewski, "Graph convolutional networks: A comprehensive review," *Comput. Social Netw.*, vol. 6, no. 1, pp. 1–23, 2019.
[43] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–12.
[44] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016, *arXiv:1607.06450*.
[45] K. Bhatia et al., "Sparse local embeddings for extreme multi-label classification," *Proc. Adv. Neural Inform. Process. Syst.*, vol. 28, 2015.
[46] R. Yan, Y. Fan, J. Zhang, J. Zhang, and H. Lin, "Service recommendation for composition creation based on collaborative attention convolutional network," in *Proc. IEEE Int. Conf. Web Serv.*, 2021, pp. 397–405.
[47] M. Liu, Y. Zhu, H. Xu, Z. Tu, and Z. Wang, "T2L2: A tiny three linear layers model for service mashup creation," in *Proc. Int. Conf. Service-Oriented Comput.*, 2021, pp. 317–331.
[48] M. Shi, Y. Tang, and J. Liu, "Functional and contextual attention-based LSTM for service recommendation in mashup creation," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 5, pp. 1077–1090, May 2019.
[49] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch geometric," in *Proc. ICLR Workshop Representation Learn. Graphs Manifolds*, 2019, pp. 1–9.
[50] S. Kornblith, M. Norouzi, H. Lee, and G. E. Hinton, "Similarity of neural network representations revisited," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 3519–3529.
[51] Y. Feng, R. Zhai, D. He, L. Wang, and B. Dong, "Transferred discrepancy: Quantifying the difference between representations," 2020, *arXiv:2007.12446*.
[52] M. Leszczynski, A. May, J. Zhang, S. Wu, C. R. Aberger, and C. Ré, "Understanding the downstream instability of word embeddings," 2020, *arXiv:2003.04983*.
[53] C. Manning and H. Schutze, *Foundations of Statistical Natural Language Processing*. Cambridge, MA, USA: MIT Press, 1999.
[54] T. N. Kipf, E. Fetaya, K. Wang, M. Welling, and R. S. Zemel, "Neural relational inference for interacting systems," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 2693–2702.
[55] S. K. Gavvala, C. Jatoth, G. Gangadharan, and R. Buyya, "QoS-aware cloud service composition using eagle strategy," *Future Gener. Comput. Syst.*, vol. 90, pp. 273–290, 2019.
[56] R. Liu, Z. Wang, and X. Xu, "Parameter tuning for S-ABCPK: An improved service composition algorithm considering priori knowledge," *Int. J. Web Serv. Res.*, vol. 16, no. 2, pp. 88–109, 2019.



Mingyi Liu received the BS degree from the School of Computer Science and Technology, Harbin Institute of Technology, in 2018. He is currently working toward the PhD degree in software engineering with the Harbin Institute of Technology (HIT), China. His research interests include service ecosystem model, service evolution analysis, data mining and knowledge graph.

Zhiying Tu (Member, IEEE) received the PhD degree in computer integrated manufacturing (Productique) from the University of Bordeaux. He is an associate professor of the School of Computer Science and Technology, Harbin Institute of Technology (HIT). Since 2013, he start to work with HIT. His research interest are service computing, enterprise interoperability, and cognitive computing. He has 20 publications as edited books and proceedings, refereed book chapters, and refereed technical papers in journals and conferences. He is the member of IEEE Computer Society, and CCF China.

Hanchuan Xu received the BSC, MSc and PhD degrees in computer science and technology from the Harbin Institute of Technology, Harbin, China, in 1999, 2003 and 2011, respectively. He is currently an associate professor with the Faculty of Computing, Harbin Institute of Technology, Harbin, China. He is the author or co-author more than 20 academic papers. His current research interests include service computing, software service engineering, enterprises computing.

Xiaofei Xu (Member, IEEE) received the PhD degree in computer science from the Harbin Institute of Technology, in 1988. He is a professor with the School of Computer Science and Technology, and vice president of the Harbin Institute of Technology. His research interests include enterprise intelligent computing, services computing, Internet of services, and data mining. He is the associate chair of IFIP TC5 WG5.8, chair of INTEROP-VLab China Pole, fellow of China Computer Federation (CCF), and the vice director of the technical committee of service computing of CCF. He is the author of more than 300 publications. He is member of the ACM.

Zhongjie Wang (Member, IEEE) received the PhD degree in computer science from the Harbin Institute of Technology, in 2006. He is a professor with the Faculty of Computing, Harbin Institute of Technology (HIT). His research interests include services computing, mobile and social networking services, and software architecture. He is the author of more than 80 publications.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:38:30 UTC from IEEE Xplore. Restrictions apply.